Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor: Dr. Ahmed Raza	**Date:**
Day:	**Semester:**

Lab 6: K-Means Clustering from Scratch

Lab Title	CLO	BT	PLO	Weightage
K-Means Clustering From Scratch				

Name	Roll Number	Report /10	Viva /5	Total /15
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

1 Learning Outcomes

After completing this lab students will be able to:

- Implement the K-Means clustering algorithm from scratch.
- Understand how different distance metrics affect clustering.
- Compare clustering results on separable vs overlapping datasets.
- Visualize cluster decision boundaries.

2 Datasets

In this lab we will experiment with two datasets.

2.1 Dataset 1: Iris Dataset

The Iris dataset contains 150 samples.

For visualization purposes we only use two features:

- Sepal Length
- Petal Length

2.2 Dataset 2: Synthetic Dataset

We will also generate a synthetic dataset using:

sklearn.datasets.make_blobs

Two versions will be created:

- Well-separated clusters
- Overlapping clusters

This allows us to observe how K-Means behaves when clusters are not clearly separable.

3 Distance Metrics

In this lab you will experiment with three distance metrics.

3.1 Euclidean Distance

$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

Produces circular cluster boundaries.

3.2 Manhattan Distance

$$d(x, y) = \sum |x_i - y_i|$$

Produces diamond-shaped cluster boundaries.

3.3 Cosine Distance

Cosine similarity measures the angle between two vectors.

$$\text{similarity} = \frac{x \cdot y}{||x||\,||y||}$$

Cosine distance:

$$d(x, y) = 1 - similarity$$

This metric is commonly used in:

- Text clustering
- Document similarity
- Embedding spaces

4 Part A: Distance Function (3 Marks)

Implement the function:

_distance(x1, x2)

The function must support the following metrics:

- Euclidean
- Manhattan
- Cosine

Use NumPy operations only.

```python
def _distance(self, x1, x2):
    """
    Compute distance between two points.

    Tasks:
    - Implement three distance metrics:
    * Euclidean: circular clusters
    * Manhattan: diamond-shaped clusters
    * Cosine: measures angle between vectors
    - Hint for cosine: distance = 1 - (dot(x1,x2)/(||x1||*||x2||))
    """
    if self.metric == "euclidean":
        diff = x1 - x2
        return np.sqrt(np.sum(diff * diff))

    elif self.metric == "manhattan":
        diff = x1 - x2
        return np.sum(np.abs(diff))

    elif self.metric == "cosine":
        dot_val = np.dot(x1, x2)

        norm1 = np.linalg.norm(x1)
        norm2 = np.linalg.norm(x2)

        if norm1 == 0 or norm2 == 0:
            return 1.0

        cos_sim = dot_val / (norm1 * norm2)
        return 1 - cos_sim

    else:
        return 0
```

5 Part B: Cluster Assignment (4 Marks)

Implement:

assign_clusters(X, centroids)

Steps:

- Compute the distance from each point to every centroid
- Assign the point to the closest centroid
- Return cluster labels

```
# ------------------------------------------------------------

def assign_clusters(self, X, centroids):
    """
    Assign each point to the nearest centroid.

    Tasks:
    - Compute distance from each point to every centroid
    - Assign label based on nearest centroid
    - Return array of cluster labels
    """
    # TODO
    labels = []

    for point in X:
        distances = []

        for centroid in centroids:
            distances.append(self._distance(point, centroid))

        labels.append(np.argmin(distances))

    return np.array(labels)
```

6 Part C: Update Centroids (4 Marks)

Implement:

update_centroids(X, labels)

Steps:

- For each cluster compute the mean of all assigned points
- Return updated centroid coordinates

```python
def update_centroids(self, X, labels):
    """
    Update centroid positions based on cluster assignments.

    Tasks:
    - For each cluster, compute mean of points assigned to it
    - If a cluster has no points (empty), randomly reinitialize
      its centroid (safeguard)
    """
    centroids = []

    for i in range(self.k):
        # TODO: get points belonging to cluster i
        cluster_points = X[labels == i]

        if len(cluster_points) == 0:
            # SAFEGUARD: choose random point as centroid
            random_index = np.random.randint(0, len(X))
            centroid = X[random_index]
        else:
            # TODO: compute mean of cluster points
            centroid = np.mean(cluster_points, axis=0)

        centroids.append(centroid)

    return np.array(centroids)
```

7 Part D: Full K-Means Algorithm (4 Marks)

Implement:

fit(X)
predict(X)

Algorithm:

1. Initialize K centroids randomly
2. Assign points to nearest centroid
3. Update centroid locations
4. Repeat until centroids converge or max iterations reached

```
def fit(self, X):
    """
    Fit the K-Means model to the data X.

    Algorithm:
    1. Initialize centroids
    2. Assign points to nearest centroid
    3. Update centroids
    4. Repeat until convergence (centroid movement < tol)
    or max iterations reached

    Tasks:
    - Implement K-Means training loop
    - Use centroid_shift to detect convergence
    """
    np.random.seed(self.random_state)
    centroids = self.initialize_centroids(X)

    for iteration in range(self.max_iter):
        labels = self.assign_clusters(X, centroids)
        new_centroids = self.update_centroids(X, labels)

        # TODO compute centroid movement
        shift = centroid_shift(centroids, new_centroids)

        centroids = new_centroids

        if shift < self.tol:
            break

    self.centroids = centroids
    self.labels_ = self.assign_clusters(X, self.centroids)
```

```python
def predict(self, X):
    """
    Predict cluster labels for new data points.

    Tasks:
    - Compute distance from each point to centroids
    - Assign each point to nearest centroid
    """
    # TODO
    labels = []

    for point in X:
        distances = []

        for centroid in self.centroids:
            distances.append(self._distance(point, centroid))

        labels.append(np.argmin(distances))

    return np.array(labels)
```

8 Part E: Boundary Visualization (Bonus)

Create a function:

plot_decision_boundaries(model, X)

Steps:

- Create a mesh grid over the feature space
- Predict cluster label for each grid point
- Plot colored decision regions

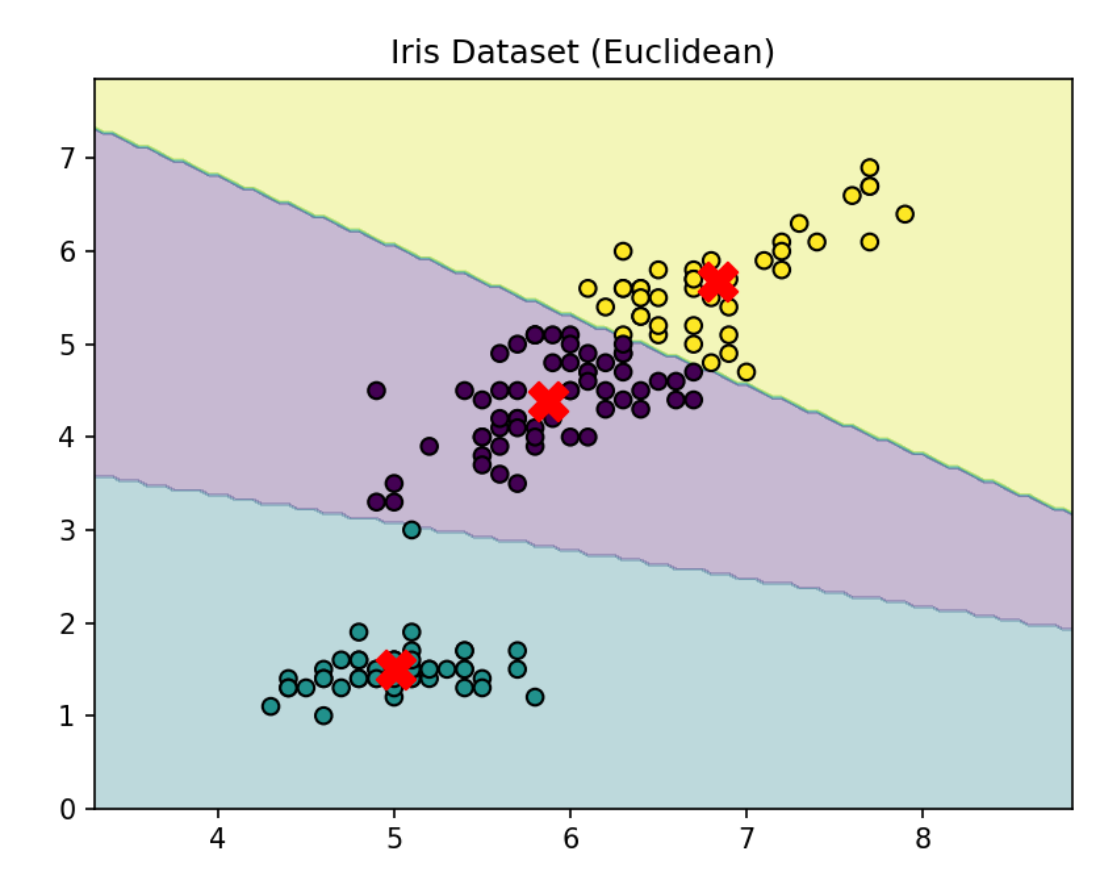
Run the visualization for:

- Euclidean distance
- Manhattan distance
- Cosine distance

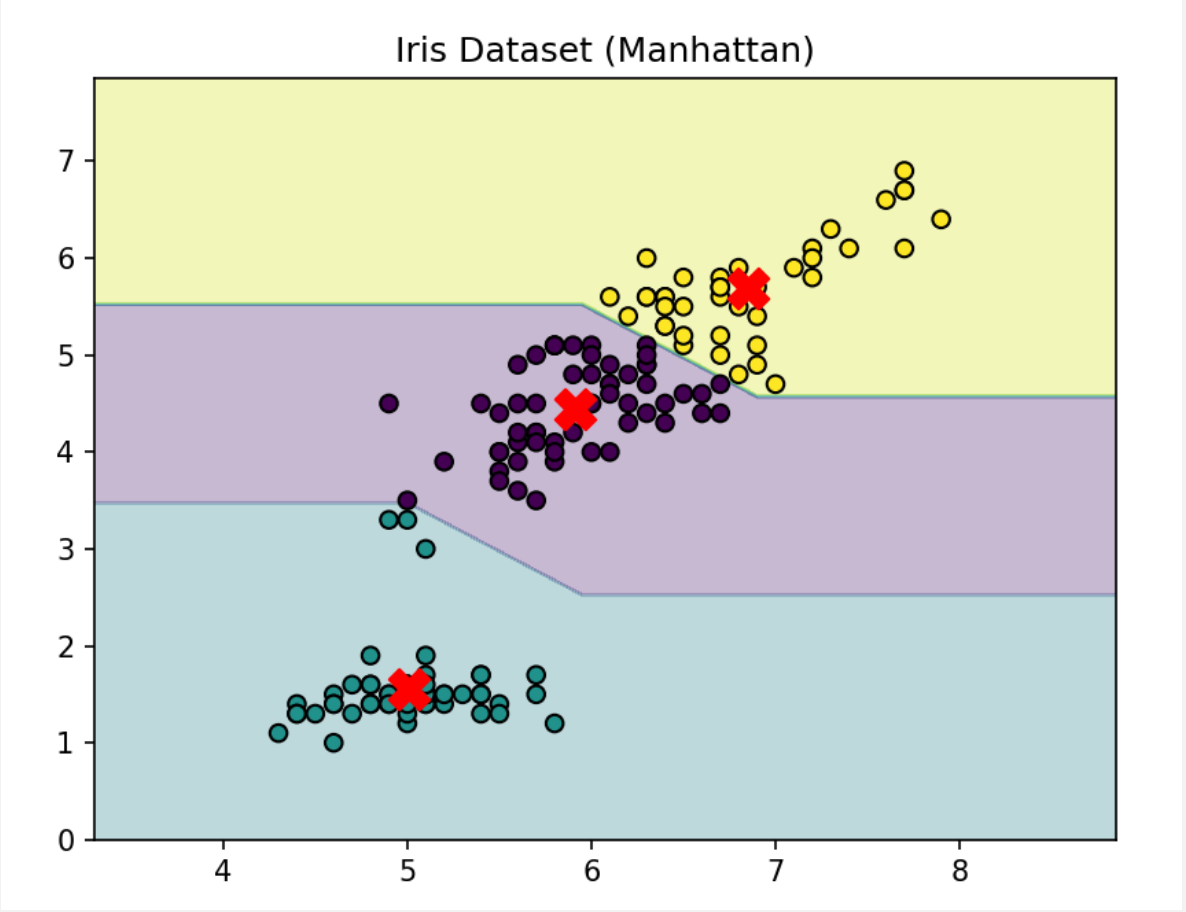
```python
def plot_decision_boundaries(model, X):
    """
    Visualize clustering regions (decision boundaries).

    Tasks:
    - Create a mesh grid over the feature space
    - Predict cluster label for each grid point
    - Plot colored decision regions
    - Useful for comparing distance metrics
    """
    x_min, x_max = X[:, 0].min()-1, X[:, 0].max()+1
    y_min, y_max = X[:, 1].min()-1, X[:, 1].max()+1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.05),
                         np.arange(y_min, y_max, 0.05))
    grid = np.c_[xx.ravel(), yy.ravel()]
    Z = model.predict(grid)
    Z = Z.reshape(xx.shape)
    plt.contourf(xx, yy, Z, alpha=0.3, cmap='viridis')
    plt.scatter(X[:, 0], X[:, 1], c=model.labels_, cmap='viridis', edgecolor='k')
    plt.scatter(model.centroids[:, 0], model.centroids[:, 1], marker='X', s=200, c='red')
    plt.title(f"Decision Boundaries ({model.metric})")
    plt.show()
```

Outputs

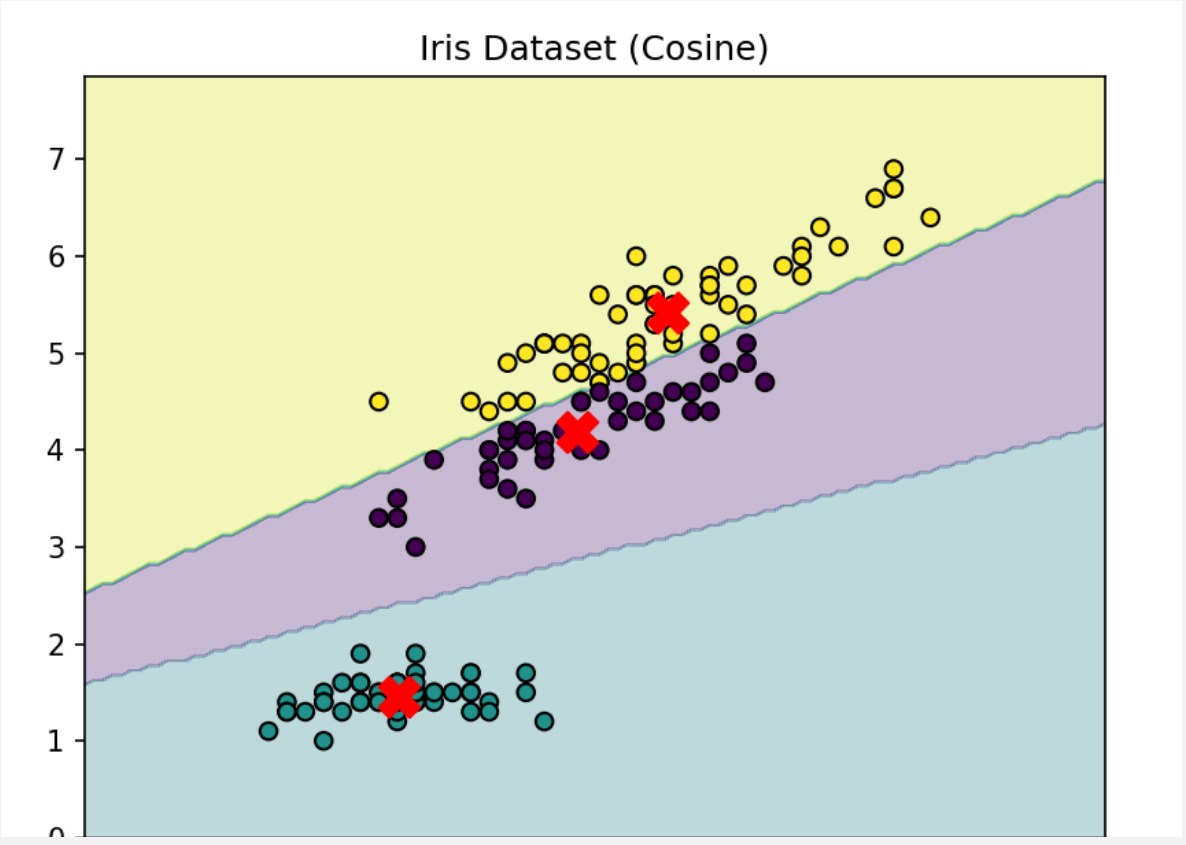
- **Visualization for Euclidean distance decision boundries**



- **Visualization for Manhattan distance decision boundaries**



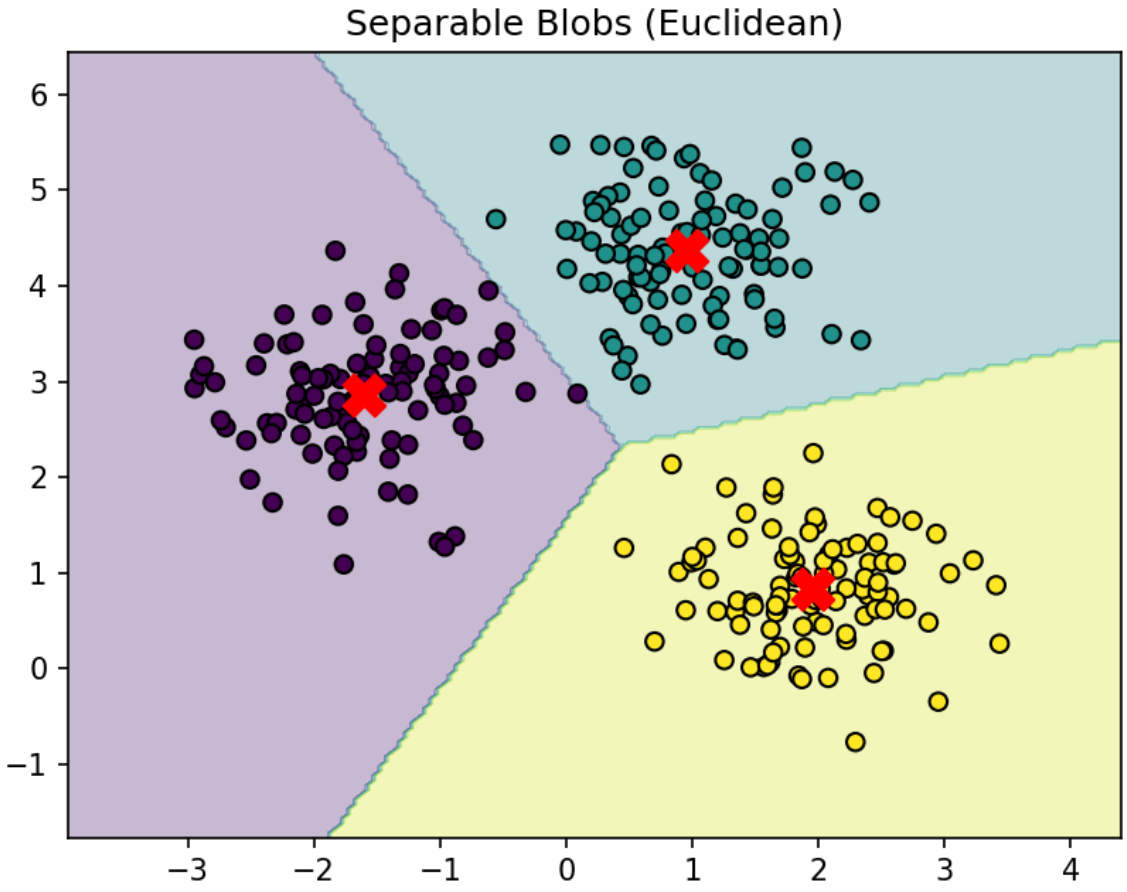
- **Visualization for Cosine distance decision boundaries**

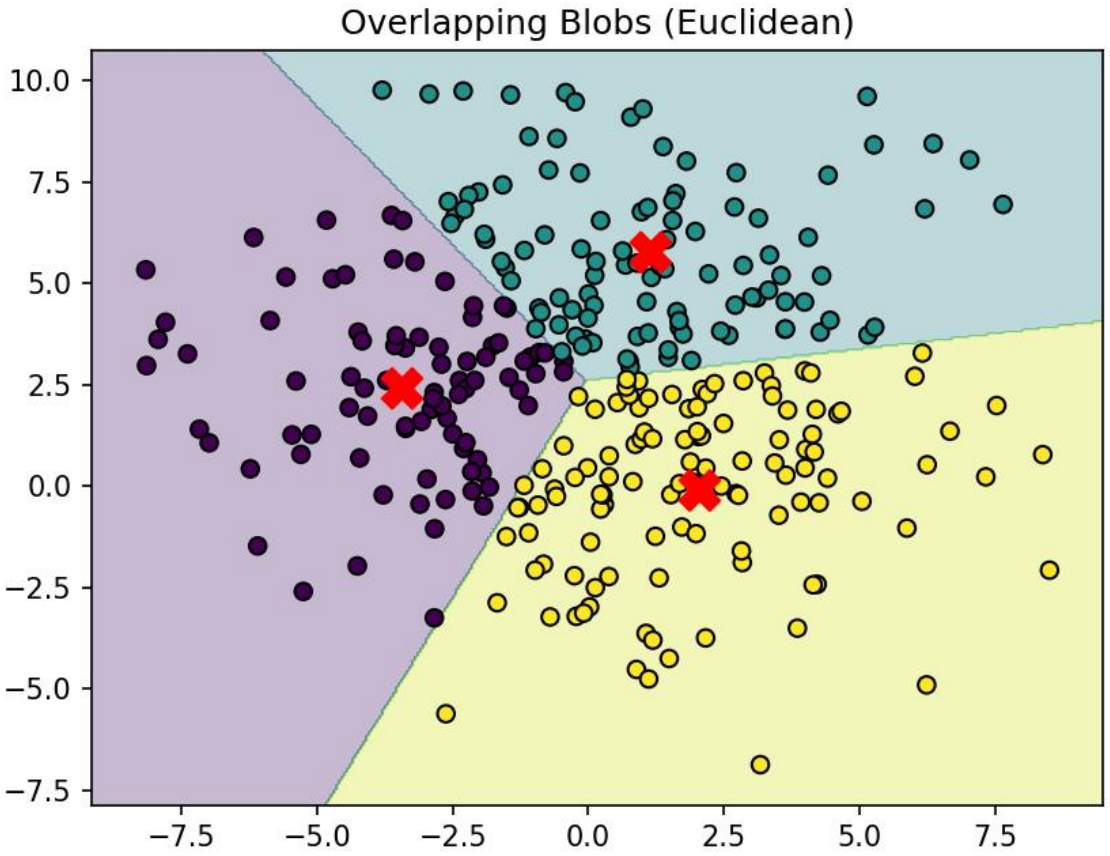


9 Part F: Experimental Comparison

Run K-Means on both datasets.

Dataset	Distance Metric	Observations
Iris	Euclidean	It creates smooth and curved decision boundaries, making clusters appear more circular in shape. Because of this, it works really well as a general-purpose method for clustering
Iris	Manhattan	It creates more angular and diagonal decision boundaries, so the clusters tend to look rectangular or diamond-shaped. It's also less affected by outliers, making it a bit more robust in noisy data.
Iris	Cosine	It creates boundaries that spread out radially from the origin, making it especially useful when the direction of the data matters more than its magnitude. For datasets with only positive values, it often gives results similar to Euclidean distance.
Separable Blobs	Euclidean	It clearly separates the clusters with well-defined boundaries, and the low inertia indicates that the data points are tightly grouped within their clusters.
Overlapping Blobs	Euclidean	It has trouble dealing with overlapping regions, so some points end up being misclassified. The higher inertia also suggests that the clusters are less compact and not as well defined.


Separable Blobs (Euclidean)
6
5
4
3
2
1
0
−1
−3
−2
−1
0
1
2
3
4


Overlapping Blobs (Euclidean)
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5

Discuss:

Question1

How do distance metrics change cluster shapes?

Answer:

Euclidean distance forms circular or spherical clusters because it measures straight-line distance the same in every direction. Manhattan distance, on the other hand, produces diamond-shaped clusters since it adds up the absolute differences along each axis. Cosine distance behaves differently,as it creates radial or angular boundaries from the origin because it focuses on the angle between vectors rather than how large they are.

Question2

Which metric performs best for overlapping clusters?

Answer:

When clusters overlap, all distance metrics tend to struggle because K-Means assumes that clusters are well-separated and have a clear, convex shape. That said, **Manhattan** distance can be a bit more resistant to outliers compared to others. Overall, K-Means isn't the best choice for overlapping data.

Question3

Does K-Means always converge to the same solution?

Answer

No, K-Means doesn't guarantee a global minimum; it usually settles for a local minimum. The final result depends heavily on where the initial centroids are placed, so different random starting points can lead to different clusterings. That's why techniques like multiple restarts or K-Means++ initialization are used to improve the chances of finding a better solution.

10 Cluster Quality: Within Cluster Sum of Squares

K-Means attempts to minimize the following objective function:

$$J = \sum_{i=1}^{n} ||x_i - \mu_{c_i}||^2$$

Where

- x_i is a data point
- μ_{c_i} is the centroid of the cluster assigned to x_i

This value is often called:

- Within Cluster Sum of Squares (WCSS) or Sum of Squared Errors (SSE)
- Inertia

Lower values indicate tighter clusters.

11 Choosing the Number of Clusters: Elbow Method

The number of clusters K is usually unknown in real applications.
The **Elbow Method** helps estimate a reasonable value for K.
Procedure:

1. Run K-Means for different values of K
2. Compute the inertia for each K
3. Plot inertia vs K

Typically the curve looks like:

High decrease → elbow → slow decrease

The “elbow” point suggests the optimal number of cluster

12 Part F: Compute Inertia (3 Marks

Implement the function:

compute_inertia(X, labels, centroids)

Steps:

- For every point compute the squared distance to its centroid
- Sum the squared distances
- Return the total inertia

```
def compute_inertia(self, X, labels, centroids):
    """
    Compute Within-Cluster Sum of Squares (WCSS / Inertia).

    Tasks:
    - For each point, compute squared distance to its centroid
    - Sum all squared distances
    - Lower values indicate tighter clusters
    """
    # TODO
    inertia = 0

    for i, point in enumerate(X):
        center = centroids[labels[i]]
        diff = point-center
        inertia += np.dot(diff, diff)

    return inertia
```

13 Part G: Elbow Method (Bonus)

Create a function:

plot_elbow_curve(X, k_values)

Steps:

- Train K-Means for multiple values of *K*
- Compute inertia for each value
- Plot inertia vs *K*

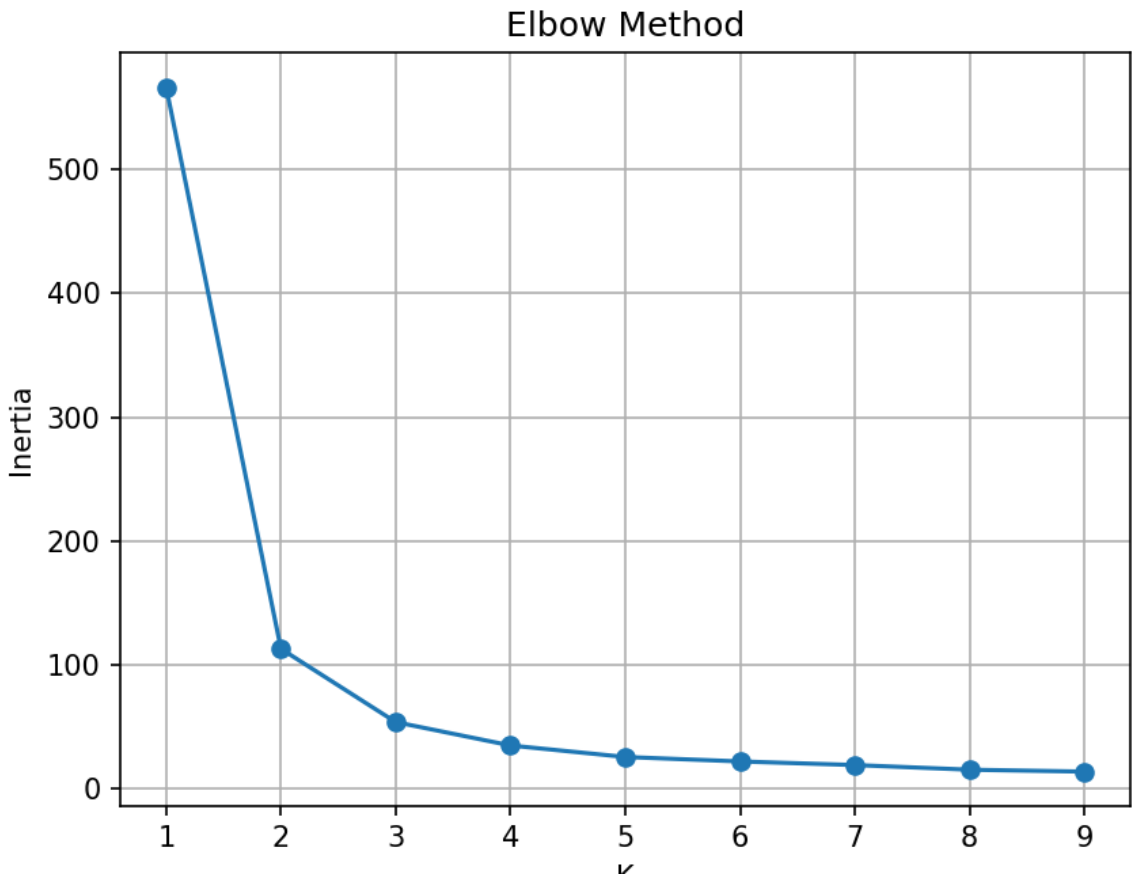
Test with:

k_values = range(1,10)

Which value of *K* appears optimal for the dataset?

```python
def plot_elbow_curve(X, k_values):
    """
    Elbow method to determine optimal K.

    Tasks:
    - Train K-Means for multiple K values
    - Compute inertia for each K
    - Plot inertia vs K
    - Observe the 'elbow' to estimate optimal clusters
    """
    inertias = []

    for k in k_values:
        model = MyKMeans(k=k)
        model.fit(X)
        labels = model.predict(X)

        inertia = model.compute_inertia(X, labels, model.centroids)
        inertias.append(inertia)

    plt.figure()
    plt.plot(k_values, inertias, marker='o')
    plt.xlabel("K")
    plt.ylabel("Inertia")
    plt.title("Elbow Method")
    plt.grid()
    plt.show()
```

Plot



Q: Which value of K appears optimal?

Ans: K = 3 seems to be the best choice for the Iris dataset. The elbow shows up at K = 3, where the drop in inertia is quite noticeable compared to K = 1 and K = 2, and after that, the improvement starts to level off. This also makes sense since the dataset naturally contains three species